CDCT in this section that hasn't already been mentioned throughout this chapter, so we'll now focus more on the PDCT approach.

3.7.2 Provider-driven contract testing

This chapter and previous chapters have detailed the "traditional" CDCT approach. For several reasons, PDCT has emerged as an option, primarily since the PactFlow team released bi-directional contract testing functionality (the terms provider-driven and bi-directional contract testing will be used interchangeably in this chapter). From our perspective, this has also been coming for a while, mainly because teams want to use the OpenAPI documentation they already have and extend it for contract testing to prevent breaking changes.

A perception of contract testing is that there is a steep learning curve and significant time investment required before seeing its value. Throughout this book, we aim to quickly break down each implementation step to provide value. However, another option to reduce the time investment and learning curve is PDCT or bi-directional contract testing by using Pact adaptors for consumers that integrate with different testing and mocking frameworks. With PDCT, providers can use their existing OpenAPI specifications as contracts, while consumers can use existing test mocks or stubs as consumer contracts. The contracts generated from mocks can then be published to the broker, like the CDCT approach. In a CDCT approach, only one contract is generated, whereas in a PDCT approach, two contracts are generated—one from the consumer and one from the provider. The PDCT life cycle is shown in figure 3.10 and explained in the following steps:

1. The consumer maintaining a JavaScript application publishes the contract to a broker.
2. Independently, the provider maintaining a Java API or event service also publishes its OpenAPI specification to the broker, which has been tested by the provider already using tools such as ReadyAPI, REST Assured, Dredd, or Postman.
3. A broker, such as PactFlow, compares the consumer contract against the provider OpenAPI specification, and the verification status is updated